Topic: Derivation of the SVM Dual Problem

1. Intuition

Support Vector Machines (SVM) aim to find a hyperplane that separates two classes with the maximum margin. Instead of directly solving for the hyperplane parameters (w, b), the dual formulation shifts the focus to identifying which training points are most important.

Only a subset of points, called **support vectors**, determine the decision boundary. The dual introduces variables α_i that measure the importance of each data point. If $\alpha_i = 0$, the point has no effect; if $\alpha_i > 0$, it is a support vector.

The key idea is that the weight vector can be expressed as a combination of training points:

$$w = \sum_i \alpha_i y_i x_i$$

Thus, the model is built entirely from the data itself rather than explicitly computing w.

2. Derivation of the Dual Problem

We start with the **primal problem** for hard-margin SVM:

$$\min_{w,b} \frac{1}{2}\|w\|^2$$

subject to:

$$y_i(w^T x_i + b) \geq 1 \quad \forall i$$

Lagrangian Formulation

Introduce Lagrange multipliers $\alpha_i \geq 0$ for each constraint:

$$L(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_i \alpha_i \big[y_i(w^T x_i + b) - 1\big]$$

Stationary Conditions

To minimize L w.r.t. w and b, take derivatives:

$$\frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_i \alpha_i y_i x_i$$

$$\frac{\partial L}{\partial b} = 0 \Rightarrow \sum_i \alpha_i y_i = 0$$

Substituting Back

Substitute $w = \sum_i \alpha_i y_i x_i$ into the Lagrangian.

First compute:

$$\|w\|^2 = \sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i^T x_j$$

Also,

$$\sum_i \alpha_i y_i w^T x_i = \sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i^T x_j$$

Using $\sum_i \alpha_i y_i = 0$, the b term vanishes.

Substituting everything:

$$L = \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i^T x_j - \sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i^T x_j + \sum_i \alpha_i$$

$$= -\frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i^T x_j + \sum_i \alpha_i$$

Dual Optimization Problem

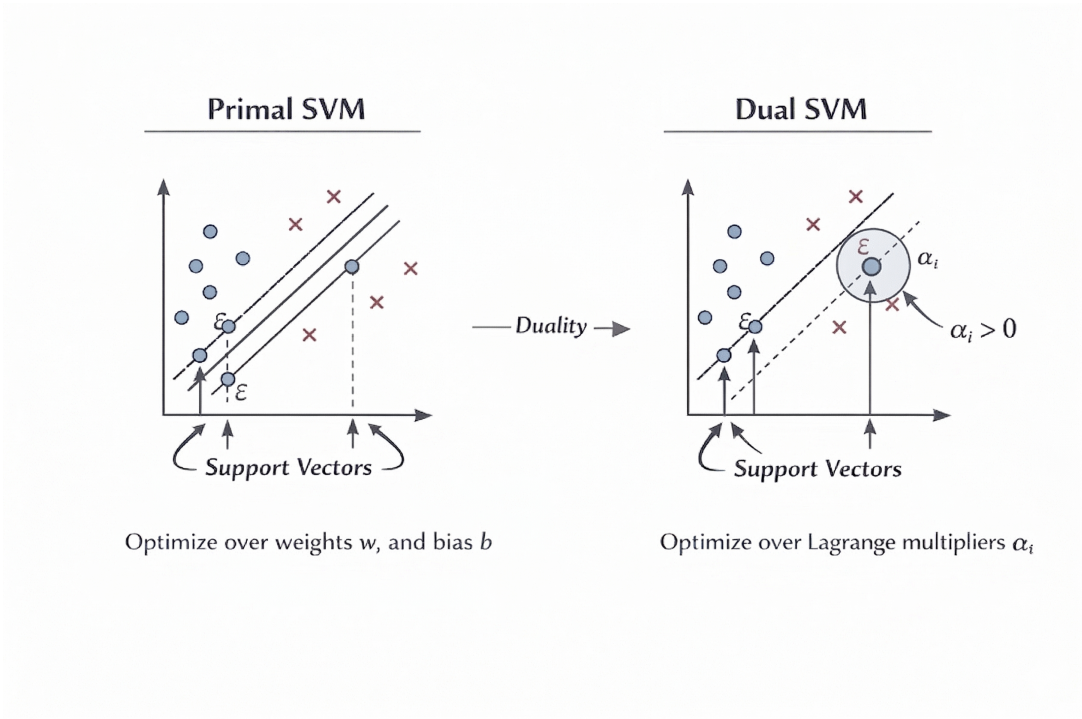
Thus, the dual becomes:

$$\max_\alpha \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i^T x_j$$

subject to:

$$\alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0$$

3. Visual Interpretation



The left side (Primal SVM) shows how the model directly finds the separating hyperplane by optimizing the weight w and bias b, while maximizing the margin. The closest points to the boundary are the **support vectors**, which determine the margin.

The right side (Dual SVM) shows that instead of solving for w and b, the model assigns importance values α_i to each data point. Only points with $\alpha_i > 0$ (support vectors) influence the decision boundary.

The duality between the two forms ensures both approaches give the same solution, but the dual formulation depends only on important points and allows the use of kernels.

4. Connections

- **Kernel Trick:** The dual formulation depends only on dot products $x_i^T x_j$. These can be replaced with kernel functions, enabling SVMs to learn non-linear decision boundaries.
- **Regularization:** Minimizing $\|w\|^2$ is equivalent to controlling model complexity, similar to L2 regularization in ridge regression.

5. Why Dual is Preferred

- **Enables the use of kernel functions for non-linear classification**
  The dual formulation depends only on dot products $x_i^T x_j$, which can be replaced by kernel functions. This allows SVMs to learn complex, non-linear decision boundaries without explicitly transforming the data.
- **Produces sparse solutions (only support vectors matter)**
  In the dual solution, most Lagrange multipliers α_i become zero. Only a small subset of points (support vectors) have $\alpha_i > 0$, making the model memory-efficient and easier to interpret.
- **Efficient in high-dimensional settings**
  When the number of features is very large, solving the primal can be expensive. The dual formulation focuses on the number of data points and uses inner products, making it more efficient in such cases.
- **Avoids explicit computation of w**
  The weight vector is expressed as a combination of training points, so we never need to compute w directly. This is especially useful when working with kernels, where w may not even exist in explicit form.

6. References

- Image in Visual Interpretation
- Prof. C.V. Jawahar Class Notes
- *Deisenroth, Faisal, Ong. Mathematics for Machine Learning*
- *Stanford CS229 Lecture Notes (SVM)*